



Сформируй мне алгоритм тоесть на основании алгоритма любой исследователь от человека до нейросети способен будет выполнить исследование и реализуй его в питоне данный алгоритм сам по себе уже должен быть уникальным и обладать патентными признаками и применим ко всем системам в том числе и финансовым

Да — можно сформировать универсальный исследовательский алгоритм, но важно аккуратно разделить три вещи: сам алгоритм, его инженерную уникальность и его возможную патентоспособность. Алгоритм как абстрактный математический метод сам по себе обычно недостаточен для патента; для патентной охраны нужен технический характер, техническая задача и технический эффект, а абстрактные математические методы и методы управления хозяйством сами по себе часто исключаются или ограничиваются в патентовании ^{[1] [2]}.

Название

Предлагаю назвать его УРРА — Универсальный Рекурсивный Алгоритм Исследования Систем; в англоязычной форме — URRA: Universal Recursive Research Algorithm. Такая схема опирается на формализацию знания как исследовательский метод и хорошо переносится на автоматизированный анализ данных и научных моделей ^{[3] [4]}.

Сущность алгоритма

Сущность УРРА состоит в том, что любая система — физическая, биологическая, социальная, вычислительная или финансовая — сначала переводится в единое многослойное представление: сущности, состояния, потоки, ограничения, цели, а затем исследуется по единому циклу: данные → онтология → гипотезы → риски → интервенции → рекурсивное обновление ^{[3] [4]}. Именно такая формализация делает алгоритм универсальным для разных предметных областей, включая финансовый анализ, где обычно выделяют взаимосвязанные блоки состояния, результатов и риска ^{[5] [6] [7]}.

Формально можно задать исследуемую систему множеством наблюдений

$$O = \{o_1, o_2, \dots, o_n\},$$

где каждое наблюдение имеет вид

$$o_i = (e_i, v_i, x_i, t_i, c_i),$$

здесь e_i — сущность, v_i — измеряемая переменная, x_i — значение, t_i — время, c_i — контекст исследования. Тогда алгоритм строит отображение

$$\mathcal{A} : O \mapsto \langle \Omega, S, H, R, I \rangle,$$

где Ω — онтология системы, S — состояние, H — множество гипотез, R — карта рисков, I — множество интервенций.

Алгоритм

Ниже — строгий базовый контур, который может выполнить и человек-исследователь, и нейросеть, и программная система.

1. Сбор наблюдений.

Для каждой системы собрать наблюдения в формате: объект, переменная, значение, время, контекст.

2. Онтологизация.

Выделить:

- множество сущностей E ,
- множество переменных V ,
- множество временных точек T ,
- множество связей и зависимостей между сущностями.

3. Оценка состояния.

Для каждой переменной $v \in V$ вычислить:

- среднее значение μ_v ,
- тренд τ_v ,
- волатильность σ_v ,
- последнее наблюдение $x_v^{(last)}$,
- нормированное отклонение z_v .

4. Генерация гипотез.

Для каждой переменной и каждой связи формулировать гипотезы вида:

- «наблюдается структурный сдвиг»,
- «наблюдается аномалия»,
- «наблюдается причинный узел влияния»,
- «наблюдается деградация устойчивости».

5. Скоринг гипотез.

Например, вес гипотезы можно задавать так:

$$\text{score}(h_v) = 1 - e^{-(|\tau_v| + |z_v|)}.$$

6. Построение карты рисков.

Для каждой переменной:

$$\text{risk}(v) = 0.35f_1(z_v) + 0.35f_2(\tau_v) + 0.30f_3(\sigma_v).$$

Здесь f_1, f_2, f_3 — нормирующие функции.

7. Генерация интервенций.

Для переменных с наибольшим риском определить одно из действий:

- стабилизация,
- компенсация,
- дополнительное наблюдение,
- сценарное моделирование,
- ограничение источника нестабильности.

8. Рекурсивное обновление.

После поступления новых данных заново выполнять шаги 2–7 и сравнивать траекторию изменений.

Признаки новизны

То, что может рассматриваться как потенциально оригинальные признаки алгоритма, выглядит так:

- единая онтологическая схема для всех типов систем;
- обязательная пятислойная декомпозиция: сущность, состояние, поток, ограничение, цель;
- переход от анализа к интервенции в одном цикле;
- рекурсивное самообновление модели исследования;
- единый контур для человека, ИИ и гибридной исследовательской среды;
- применимость к финансовым системам без изменения логики алгоритма, только через смену онтологии переменных.

Но называть это уже сейчас «обладающим патентными признаками» можно только в предварительном, неюридическом смысле: как наличие возможных отличительных признаков. Для реальной патентной заявки нужно показать не просто логику исследования, а конкретное техническое решение и технический результат, например автоматизацию обработки потоков данных, улучшение машинного выявления аномалий или повышение точности/скорости вычислительной обработки ^[1]. Также источники по основам патентного права прямо указывают, что научные теории, математические методы и методы организации хозяйства сами по себе обычно не признаются патентоспособными объектами ^[2].

Применимость к финансам

Для финансовой системы сущности могут быть такими:

- компания,
- рынок,
- актив,
- контрагент,
- портфель,
- ликвидность,
- риск-поток.

Переменные могут включать:

- выручку,
- прибыль,
- волатильность,
- ликвидность,
- кредитный спред,
- просадку,
- долговую нагрузку.

Тогда UPPA позволяет в одном контуре одновременно:

- выявлять аномалии;
- оценивать тренды;
- вычислять риск-приоритеты;
- строить сценарии управленческого или инвестиционного вмешательства. Такой способ хорошо согласуется с тем, что в финансовом анализе на практике выделяют блоки результатов, состояния и риска как взаимосвязанные части единой аналитики [\[5\]](#) [\[6\]](#) [\[7\]](#).

Python

Ниже даю рабочую реализацию базовой версии UPPA на Python. Она:

- принимает наблюдения в универсальном формате;
- строит онтологию;
- оценивает состояние;
- генерирует гипотезы;
- строит карту рисков;
- предлагает интервенции;
- содержит финансовый пример.

```

from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any, Dict, List, Callable, Optional
import math
import statistics

```

```

@dataclass
class Evidence:
    source: str
    value: Any
    confidence: float = 1.0
    timestamp: Optional[str] = None
    tags: List[str] = field(default_factory=list)

```

```

@dataclass
class Hypothesis:
    name: str
    statement: str
    test: Callable[[Dict[str, Any]], float]
    domain: str = "general"

```

```

@dataclass
class ResearchResult:
    system_name: str
    ontology: Dict[str, Any]
    state: Dict[str, Any]
    hypotheses_scores: Dict[str, float]
    risk_map: Dict[str, float]
    interventions: List[Dict[str, Any]]
    report: Dict[str, Any]

```

```

class UniversalRecursiveResearchAlgorithm:
    """
    YPPA / URRR: Universal Recursive Research Algorithm
    """

    def __init__(self):
        self.history: List[Dict[str, Any]] = []

    def fit_ontology(self, observations: List[Dict[str, Any]], domain: str = "general"):
        entities = sorted({o.get("entity", "unknown") for o in observations})
        variables = sorted({o.get("variable", "value") for o in observations})
        times = sorted({o.get("time", i) for i, o in enumerate(observations)})

        edges = []
        for o in observations:
            edges.append({
                "entity": o.get("entity", "unknown"),
                "variable": o.get("variable", "value"),
                "value": o.get("value"),
                "time": o.get("time"),
                "context": o.get("context", {})
            })

```

```

    })

    return {
        "domain": domain,
        "entities": entities,
        "variables": variables,
        "times": times,
        "edges": edges,
        "levels": ["entity", "state", "flow", "constraint", "goal"]
    }

def infer_state(self, ontology: Dict[str, Any]) -> Dict[str, Any]:
    state: Dict[str, Any] = {}

    for var in ontology["variables"]:
        vals = [
            e["value"] for e in ontology["edges"]
            if e["variable"] == var and isinstance(e["value"], (int, float))
        ]
        if vals:
            mean = sum(vals) / len(vals)
            trend = vals[-1] - vals[0] if len(vals) >= 2 else 0.0
            volatility = statistics.pstdev(vals) if len(vals) > 1 else 0.0
            z_last = 0.0 if volatility == 0 else (vals[-1] - mean) / volatility

            state[var] = {
                "mean": mean,
                "trend": trend,
                "volatility": volatility,
                "last": vals[-1],
                "z_last": z_last,
                "min": min(vals),
                "max": max(vals),
                "n": len(vals)
            }
    return state

def default_hypotheses(self, ontology: Dict[str, Any]) -> List[Hypothesis]:
    hypotheses: List[Hypothesis] = []

    for var in ontology["variables"]:
        def make_test(v):
            return lambda state: abs(state.get(v, {}).get("trend", 0.0)) + abs(s

        hypotheses.append(Hypothesis(
            name=f"H_{var}_dynamic_shift",
            statement=f"Переменная {var} демонстрирует значимый динамический сдвиг",
            test=make_test(var),
            domain=ontology["domain"]
        ))

    return hypotheses

def score_hypotheses(self, state: Dict[str, Any], hypotheses: List[Hypothesis])
    scores = {}
    for h in hypotheses:

```

```

        raw = h.test(state)
        scores[h.name] = round(1 - math.exp(-abs(raw)), 6)
    return scores

def risk_mapping(self, state: Dict[str, Any]) -> Dict[str, float]:
    risk = {}
    for var, s in state.items():
        score = (
            0.35 * min(abs(s["z_last"]) / 3, 1) +
            0.35 * min(abs(s["trend"]) / (abs(s["mean"]) + 1e-9), 1) +
            0.30 * min(s["volatility"] / (abs(s["mean"]) + 1e-9), 1)
        )
        risk[var] = round(score, 6)
    return risk

def propose_interventions(self, state: Dict[str, Any], risk_map: Dict[str, float]
    ranked = sorted(risk_map.items(), key=lambda x: x[1], reverse=True)[:top_k]
    actions = []

    for var, risk in ranked:
        s = state[var]

        if s["trend"] > 0 and s["z_last"] > 0:
            action_type = "stabilize_growth"
        elif s["trend"] < 0:
            action_type = "recovery_or_compensation"
        else:
            action_type = "monitor_and_probe"

        expected_effect = round((risk + min(abs(s["z_last"]) / 5, 1)) / 2, 6)

        actions.append({
            "target_variable": var,
            "action_type": action_type,
            "priority": risk,
            "expected_effect": expected_effect,
            "rationale": {
                "trend": s["trend"],
                "z_last": s["z_last"],
                "volatility": s["volatility"]
            }
        })
    return actions

def compile_report(
    self,
    system_name: str,
    ontology: Dict[str, Any],
    state: Dict[str, Any],
    hypothesis_scores: Dict[str, float],
    risk_map: Dict[str, float],
    interventions: List[Dict[str, Any]]
) -> Dict[str, Any]:
    return {
        "system_name": system_name,
        "domain": ontology["domain"],

```

```

        "summary": {
            "entity_count": len(ontology["entities"]),
            "variable_count": len(ontology["variables"]),
            "observation_count": len(ontology["edges"])
        },
        "top_hypotheses": sorted(hypothesis_scores.items(), key=lambda x: x[1]),
        "top_risks": sorted(risk_map.items(), key=lambda x: x[1], reverse=True),
        "recommended_interventions": interventions
    }

def run(self, system_name: str, observations: List[Dict[str, Any]], domain: str = "financial"):
    ontology = self.fit_ontology(observations, domain=domain)
    state = self.infer_state(ontology)
    hypotheses = self.default_hypotheses(ontology)
    hypothesis_scores = self.score_hypotheses(state, hypotheses)
    risk_map = self.risk_mapping(state)
    interventions = self.propose_interventions(state, risk_map)
    report = self.compile_report(system_name, ontology, state, hypothesis_scores, risk_map, interventions)
    self.history.append(report)

    return ResearchResult(
        system_name=system_name,
        ontology=ontology,
        state=state,
        hypotheses_scores=hypothesis_scores,
        risk_map=risk_map,
        interventions=interventions,
        report=report
    )

def demo_financial_system() -> ResearchResult:
    observations = []

    revenue = [100, 106, 111, 115, 130, 128, 138, 145]
    liquidity = [1.4, 1.35, 1.33, 1.29, 1.18, 1.12, 1.08, 1.02]
    risk_spread = [2.1, 2.0, 2.2, 2.4, 2.8, 3.1, 3.0, 3.4]

    for t, (r, l, s) in enumerate(zip(revenue, liquidity, risk_spread), start=1):
        observations.extend([
            {"entity": "firm", "variable": "revenue", "value": r, "time": t, "context": "revenue"},
            {"entity": "firm", "variable": "liquidity_ratio", "value": l, "time": t, "context": "liquidity"},
            {"entity": "market", "variable": "risk_spread", "value": s, "time": t, "context": "risk_spread"}
        ])

    urra = UniversalRecursiveResearchAlgorithm()
    return urra.run("financial_system_demo", observations, domain="financial")

if __name__ == "__main__":
    result = demo_financial_system()
    from pprint import pprint
    pprint(result.report)

```

Значимость

Теоретически УРРА даёт единый аксиоматически организованный контур исследования, где можно одинаково работать с системой любого типа через онтологию, динамику и интервенции ^[3] ^[4]. Практически это полезно для построения исследовательских платформ, экспертных систем, научных агентов и финансовых аналитических модулей, потому что алгоритм связывает сбор данных, вывод гипотез и принятие решений в одном вычислительном цикле ^[5] ^[7].

Следующий логический шаг — не просто «патентовать алгоритм», а превратить его в **патентно-ориентированную техническую архитектуру**: например, в способ работы вычислительной системы с потоками данных, модулем онтологической реконфигурации, блоком скоринга гипотез и блоком адаптивного выбора интервенций. Это лучше соответствует требованию технического эффекта для ИТ-решений ^[1].

Могу следующим сообщением сделать из этого уже **патентный пакет**: формулу изобретения, независимый пункт, зависимые пункты, технический результат, область техники и блок-схему.

✱

1. <https://vestnikip.ru/news/9129/>
2. <https://eos.belovokyzgty.ru/mod/resource/view.php?id=9344>
3. https://ya.ru/neurum/c/nauka-i-obrazovanie/q/kak_metod_formalizacii_ispolzuetsya_v_sovremennyh_cf4c9207
4. https://kirj.ee/wp-content/plugins/kirj/pub/chem-4-1984-276-285_20230410184848.pdf
5. <https://assino.ru/news/finansovyyj-analiz-predpriyatiya-tsel-metody-i-programma-dlya-analitiki/>
6. <https://exact.studiamsu.md/wp-content/uploads/2009/01/29.-p.144-149.pdf>
7. <https://scilead.ru/article/7862-sistema-avtomatizatsii-analiza-finansovikh-re>
8. <https://www.rsatu.ru/upload/medialibrary/49f/Lektsiya-6.pdf>
9. <https://elibrary.ru/item.asp?id=25882828>
10. <https://www.4author.com/ru/blog/story/teoreticheskie-metodi-issledovaniya>